

Dataset

Each dialogue contains n audio files named `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav`. The chunks are individual turns. The filenames correspond only to the shuffled order. They do not indicate where a chunk belongs in the original conversation. Each dialogue has 7–20 chunks, mono, 44.1 kHz (you may resample).

`prefix.json` contains the filename indexes of the first two chunks in each dialogue. This identifies the true beginning of the dialogue and removes the ambiguity between reading the conversation forward or backward.

For example: 11: [7, 12] means that the first and second turns of dialogue 11 are `chunk_7.wav` and `chunk_12.wav`, respectively.

What you get

You receive **two folders in identical format**:

Folder	Dialogues	<code>answers.json</code> ?	Use it to
<code>dataset/train/</code>	1,288	✓ included	train / fine-tune your model
<code>dataset/test_public/</code>	100	✓ included	run your pipeline and self-score locally

During grading time, your `dataset/test_public/` folder is transparently replaced by a hidden evaluation set (`test_leaderboard_a` for the public leaderboard and `test_leaderboard_b` for the final leaderboard) — these have the same size and format as `dataset/test_public/` but without `answers.json`.

Your notebook is executed again on that data, and the `answers.json` file it produces is used for scoring. The held-out test dialogues come from the same distribution as `train`, so your local `test_public` score is a faithful preview.

Directory structure

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

Output

For each dialogue, determine the original chronological order of its audio chunks. Your prediction should be a permutation P of $\{0, 1, \dots, n-1\}$, where $P[i]$ is the predicted chronological position of `chunk_i.wav` (0 = first).

Your output file `answers.json` should map each dialogue ID to its predicted permutation:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

Example

A dialogue has 3 shuffled chunks `chunk_0`, `chunk_1`, `chunk_2`:

shuffled chunk	spoken content	true position (rank)
<code>chunk_0.wav</code>	<i>"No worries — I'll send you the notes afterwards."</i>	2 (last)
<code>chunk_1.wav</code>	<i>"Hey, are you coming to the three o'clock meeting?"</i>	0 (first)
<code>chunk_2.wav</code>	<i>"I can't — I've got a dentist appointment then."</i>	1

True order is **chunk_1** → **chunk_2** → **chunk_0**, so $P = [2, 0, 1]$, and `prefix.json` holds `[1, 2]`.

 **P must be a genuine permutation:** length n , 0-indexed, each value exactly once. Duplicates, missing values or out-of-range entries (e.g. 1-indexed) score 0 for that dialogue, as does a dialogue missing from the file. A malformed or non-JSON file is rejected.

Scoring

The scoring for this task is **pairwise ordering accuracy**. It checks every pair of chunks and asks: *which of the two should come first?* A pair is correct if your prediction gives the same answer as the ground truth. For a dialogue with n chunks there are

$$M = n(n - 1)/2$$

pairs; let I be the number of inversions — pairs ordered differently from the ground truth:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

The final score is the average of per-dialogue scores over all dialogues in the split.

Allowed models

You can only use the following pre-trained models to solve this task, both during the training and evaluation. All of these models are already downloaded and available in the environment. You can see examples of how to use them in baseline notebook `solution.ipynb`. Please note you cannot use any other model, and your program has no internet access.

- **Speech representations: wav2vec 2.0.** The **Whisper encoder** may also be used as a feature extractor. [wav2vec model card](#)
- **Automatic speech recognition (ASR): OpenAI Whisper** (any size). [Whisper model card](#)
- **Language model: Qwen2.5-0.5B**, which may be used either zero-shot or fine-tuned on the provided `train` split. [Qwen model card](#) Note that the 10-minute limit must cover any training or fine-tuning you do at grade time plus inference on the evaluation set.

How to submit

- Open `solution.ipynb` and run all cells. Confirm it writes `answers.json` in the working directory with a permutation for every dialogue in `dataset/test_public/` (100 dialogues). At grade time the notebook is re-run on the hidden test set and the `answers.json` it produces there is scored.
- Improve the solution if you want — or don't; the baseline alone validates the pipeline.

- Open the Git tab in the left sidebar of JupyterLab.
- **Stage** `solution.ipynb` (the + icon next to it).
- Enter a commit message and click **Commit**.
- Click the cloud-with-up-arrow to push.
- Return to this Contest page and click **Submit**.

Submit exactly one file, named `solution.ipynb`.